

Cache-Aware Memory Layout for Behavior Trees

Minimizing Instruction Cache Misses
via Minimum Linear Arrangement

MinLA / NP-Hard

MILP + Metaheuristics

Embedded Systems

Victor Oliveira · victor.rsoliveira@gmail.com

Context: BTreeFy Runtime

What is BTreeFy?

- › C library for **Behavior Trees** on embedded targets (bare-metal, POSIX, Zephyr)
- › Nodes stored as a **flat 1D C array** of `btf_node` structs
- › Traversal is DFS — pointer jumps follow `child`, `sibling`, `parent` index fields

The `btf_node` layout in memory

```
struct btf_node {  
    uint32_t parent;  
    uint32_t child;  
    uint32_t sibling;  
    btf_action_fn_t action;  
    btf_control_fn_t control;  
    char *name;  
};
```

Array index → memory address

KEY The **order** in which nodes appear in the array directly determines **spatial locality** during DFS traversal.

Behavior Trees — Quick Primer

Two node types

Control nodes (internal) — define policy:

- › **Sequence** ①②③: run children left→right; stop on first FAILURE
- › **Fallback** ①②③: run children left→right; stop on first SUCCESS

Leaf nodes — actual work:

- › **Action**: executes something; returns SUCCESS / FAILURE / RUNNING

- › **Condition**: evaluates a predicate; no side-effects

5-node example (BTreeFy)

```
Fallback [idx 0]
├─ Sequence [idx 1]
│   ├── Action A [idx 2]
│   └─ Action B [idx 3]
└─ Action C [idx 4]
```

In BTreeFy each node stores **three integer indices**:
parent, child, sibling — pointers into the flat C array.

KEY The order in which nodes appear in the flat C array directly determines **spatial locality** during execution.

BT Execution – The DFS Tick

Node access order (one tick)

1. load [0] ← Fallback: enter
2. load [1] ← Sequence: enter
3. load [2] ← Action A → SUCCESS
4. load [1] ← Sequence: re-entered!
5. load [3] ← Action B → SUCCESS
6. load [1] ← Sequence: re-entered!
→ returns SUCCESS
7. load [0] ← Fallback: re-entered!
→ done

The locality problem

Access sequence as array indices:

[0, 1, 2, 1, 3, 1, 0]

- › Control nodes are **re-accessed repeatedly**
- › Each access loads the node's cache line
- › If node and its **child or sibling are far in the array** → cache eviction → **reload cost**

Insight: the CPU pays for distance — not just traversal. Nodes connected by an edge should sit **close together in the array**.

Toy Example — Why Layout Matters

Graph edges — LCRS: collect all non-null parent, child, sibling fields across every node: $E = \{(0, 1), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3)\}$ — **6 edges**, all weights $W = 1$

Layout A — DFS pre-order (parser output)

Array: $[F, S, A, B, C] \rightarrow y = [0, 1, 2, 3, 4]$

$$C_A = |0 - 1| + |0 - 4| + |1 - 2| + |1 - 3| + |1 - 4| + |2 - 3|$$

$$= 1 + 4 + 1 + 2 + 3 + 1 = \mathbf{12}$$

S has degree 4 but sits at index 1 — large distances to C (dist 3) and B (dist 2).

Layout B – optimised ✦ provably optimal

Array: [C, F, S, A, B] $\rightarrow$ $y = [0, 1, 2, 3, 4]$

$$C_B = |1 - 2| + |1 - 0| + |2 - 3| + |2 - 4| + |2 - 0| + |3 - 4|$$

$$= 1 + 1 + 1 + 2 + 2 + 1 = \mathbf{8}$$

S centred at index 2; pairs C - F and A - B are adjacent.

RESULT Layout B achieves a **33% cost reduction**. Cost 8 is **provably optimal**: S has degree 4, so ≥ 2 of its edges must span ≥ 2 slots, giving lower bound = 8.

Modelling the BT as a Graph $G = (V, E, W)$

Vertex set

$V = \{0, 1, \dots, n - 1\}$ — one vertex per BT node

Directly the index of the node in the `bt f_node[]` array.

Edge set

For each node i , add an undirected edge for each non-null pointer field:

- › $(i, \text{node.parent})$ if `parent` $\neq$ NULL

- › $(i, \text{node.child})$ if $\text{child} \neq \text{NULL}$
- › $(i, \text{node.sibling})$ if $\text{sibling} \neq \text{NULL}$

Duplicates removed. Result: $|E| = O(n)$ for a tree.

Edge weight

W_{ij} = **traversal frequency** of edge (i, j) — how many times per tick the CPU moves between nodes i and j .
Uniform $W = 1$ for the baseline (Week 1); profile-guided weights for later phases.

Minimum Linear Arrangement (MinLA)

Problem statement

Find a bijection $y : V \rightarrow \{0, 1, \dots, n - 1\}$ that minimises:

$$\min_y \sum_{(i,j) \in E} W_{ij} \cdot |y_i - y_j|$$

subject to: $y_i \neq y_j \quad \forall i \neq j$

(all slots distinct)

y_i

Memory slot assigned to node i .

The **decision variable**: what

position in the C array does
node i occupy?

$$|y_i - y_j|$$

Distance between nodes i and j
in the array. Proxy for cache line
separation.

$$W_{ij}$$

Weight of edge (i, j) — how
often the CPU traverses this
link. High weight $\rightarrow$ must be
close.

COMPLEXITY

MinLA is **NP-Hard** for general graphs (Garey & Johnson, 1979). BT graphs are trees — MinLA on trees is polynomial, but weighted and with cache-model extensions it re-introduces hardness.

MILP Formulation — Linearising the Problem

Step 1 — Eliminate the absolute value

Replace $|y_i - y_j|$ with auxiliary variable $d_{ij} \geq 0$:

$$d_{ij} \geq y_i - y_j \quad \forall (i, j) \in E$$

$$d_{ij} \geq y_j - y_i \quad \forall (i, j) \in E$$

A minimising solver will drive d_{ij} to **exactly** $|y_i - y_j|$.
Objective becomes: $\min \sum W_{ij} d_{ij}$ — now **linear**.

Step 2 — Enforce unique slots (Big-M)

Introduce binary $z_{ij} \in \{0, 1\}$ encoding which of i, j comes first:

$$y_i - y_j \geq 1 - Mz_{ij} \quad \forall i \neq j$$

$$y_j - y_i \geq 1 - M(1 - z_{ij}) \quad \forall i \neq j$$

If $z_{ij} = 0$: forces $y_j - y_i \geq 1$ (i.e. i comes before j). If $z_{ij} = 1$: forces $y_i - y_j \geq 1$ (i.e. j before i). $M = n$ suffices as Big-M.

COMPLEXITY $O(n)$ continuous vars $(y_i, d_{ij}) + O(n^2)$ binaries (z_{ij}) . Binary count is the **exponential bottleneck** — motivates heuristics for large n .

The Exponential Wall (Week 2)

Why MILP alone is not enough

- › No-overlap constraints introduce $O(n^2)$ binary variables z_{ij}
- › Branch-and-bound search space grows **exponentially** with n
- › CBC hits intractability near $n \approx 15-20$
- › Real BTs: 20–200 nodes → **MILP alone infeasible**

Week 2 deliverable

Empirical scaling study:

- › $n \in \{5, 8, 10, 12, 15, 20\}$, 5 random BTs each
- › Record: solver time, optimality gap at timeout, memory
- › Plot **solver time vs. n** to locate the exact failure point
- › This **motivates heuristics** — the core scientific contribution

Reference: Petit (2011) — MinLA benchmark experiments

Heuristic Solution: Simulated Annealing (Week 3)

Algorithm Design

- › **Neighbourhood**: swap positions $y_a \leftrightarrow y_b$ for random pair (a, b)
- › **Cooling**: geometric schedule $T_{k+1} = \alpha \cdot T_k$, $\alpha = 0.995$
- › **Stop**: $T < T_{\min} = 10^{-4}$ or iteration budget
- › **Incremental cost**: only recompute edges incident to swapped nodes — $O(k)$ per step

Initialisation: Spectral Sequencing

- › Compute **Fiedler vector** (2nd eigenvector of graph Laplacian L)
- › Sort nodes by Fiedler value $\rightarrow$ initial permutation
- › Proven to approximate MinLA (Pettis & Hansen, 1990 — spatial locality principle)
- › Gives SA a **much warmer start** than random

TUNING Sensitivity analysis on $\alpha \in \{0.99, 0.995, 0.999\}$ — must be included in Week 3 to justify the final schedule choice.

Validation: Optimality Gap (Week 4)

Optimality Gap metric

$$\text{Gap}(\%) = \frac{C_{\text{SA}} - C_{\text{MILP}}}{C_{\text{MILP}}} \times 100$$

- Compare SA vs MILP on **all instances where MILP terminates** (time limit 5 min)
- Target: Gap < 10% on benchmark BTs

Cache simulation

- › Translate layout permutation to **memory access sequence** under DFS

- › Count misses using a **direct-mapped cache model** (parameterised by line size)
- › Validate: lower MinLA cost $\Rightarrow$ fewer simulated cache misses

Parser extension

`btfgroot_parser.py --layout <permutation>` — re-emits the `btfg_node` C array **in the optimised slot order**, closing the loop from algorithm to deployed code.

30-Day Implementation Sprint

Week Milestone

W1 MILP Formulation

W2 Scaling Analysis

W3 SA Heuristic

W4 Validation

Critical Deliverable

Correct $Ax \leq b$ structure; optimal layout for $n \leq 12$

Empirical “Exponential Wall” plot; documented failure point

SA with spectral init; α sensitivity analysis

Optimality Gap table; cache miss simulation; parser extension

STACK Python (PuLP/CBC) for MILP · NumPy/SciPy for spectral init · Matplotlib for plots · C/BTreeFy for end-to-end validation

Expected Contributions

Scientific

- › First application of **MinLA** to embedded Behavior Tree layout
- › Empirical characterisation of the **Exponential Wall** for BT-structured graphs
- › Quantified **Optimality Gap** for spectral-seeded SA on BT graphs
- › Formal link between MinLA cost and **instruction cache miss count**

Practical

- › Extended `btf_groot_parser.py` → **optimised C array output**
- › Drop-in layout optimiser for any **BTreeFy** application
- › Reproducible benchmarks on `porta_automatica` and `AssetTracking` BT models
- › Open-source under the existing BTreeFy repository

Pettis & Hansen (1990) · Petit (2011) · Garey & Johnson (1979)